

Implementation Plan for VC User Portal and Admin Enhancements in PTX

Introduction

The PTX project is a **portfolio-wide hiring platform** that already exposes a FastAPI backend and a minimal React frontend. The existing code base provides endpoints for authentication and basic CRUD operations on jobs. It also contains polished demo templates in the `amis-talent-matching` repository, such as the `demo-portal.html`, which showcase an attractive, dark-themed jobs page. Our goal is to turn those demos into production features and support a new type of user – **VC users** (venture capital fund/platform owners) – who have their own job portals.

This plan describes the changes required in the backend (FastAPI), the database (MongoDB), and the React frontend to implement the following features:

1. **Admin portal** – admins can log in, list users, create/update/delete users and tenants (VC funds), assign roles and manage each tenant's job portal.
2. **Registration wizard** – a multi-step form at `/registration` that replaces the “Create your DEMO account” mock-up with a real registration process. It creates both the tenant record and the first user (the VC administrator) and returns the unique portal URL.
3. **Unique job portal for every VC user** – each tenant gets a portal (similar to the `demo-portal.html`) where jobs from their portfolio companies are listed; jobs can be uploaded via CSV by the tenant or an admin. Uploading a new CSV replaces the existing jobs.
4. **Best-practices data storage** – extend the MongoDB schema with a `tenants` collection, index the relevant fields, and ensure multi-tenancy isolation.
5. **Provide full code** – ready-to-copy modules for backend routes, Pydantic models, React pages/components and a CSV template.

1. Review of the existing PTX code base

Backend

The PTX backend uses **FastAPI** and **MongoDB**. Authentication is implemented in `backend/app/api/routers/auth.py` with JWT tokens. The `/auth/login` endpoint validates a user and returns a JWT:

- The endpoint accepts `email` and `password` and looks up the user in the `users` collection; if the password matches, a token containing the user ID, role and tenant ID is returned ¹.

User creation is currently limited to admins. The `/auth/register` endpoint (decorated with `Depends(require_admin)`) creates a user document after checking that the role is one of `recruiter`, `analyst`, `candidate` or `admin` ². The password is hashed via PassLib.

Jobs are handled by `backend/app/api/routers/jobs.py`. A recruiter can create a job via a POST to `/jobs`; the job document includes tenant ID, external job ID, title, description, city, skills, profession and remote flag and is timestamped on insert ³. There are endpoints to fetch a single job, list jobs for

the current tenant, update a job and delete a job ⁴. The router uses dependency functions (`require_role`, `require_tenant`) to ensure the user's role matches and that they are scoped to the correct tenant.

Database indexes are created in `backend/app/core/database.py`. Unique indexes exist on the `users.email` and composite indexes on `jobs` and `candidates` for `(tenant_id, external_job_id)` and `(tenant_id, external_candidate_id)` respectively ⁵. This ensures job IDs are unique per tenant.

Frontend

The PTX frontend is a minimal React application. There is an `AdminDashboard.tsx` page that fetches basic stats but does not yet manage users ⁶. Recruiters can view jobs via `RecruiterJobsPage.tsx` ⁷. The repository does not yet include a registration page or a polished portal.

Demo templates (amis-talent-matching)

The `amis-talent-matching` repository contains high-fidelity HTML pages used for marketing demos. The `demo-portal.html` template defines a dark-themed job portal with CSS variables for colours such as `--primary`, `--accent`, `--bg`, etc. ⁸. The hero section has a gradient headline and statistics about open positions ⁹. The page features a flexible filter bar and a card grid for jobs ¹⁰. We will use this styling as inspiration for the real portal.

2. Database schema changes

To support VC users and per-tenant portals, extend the MongoDB schema as follows:

1. **tenants collection** – each VC organisation/fund is a tenant.

```
{
  id: ObjectId,
  name: String,      // name of the fund (e.g., "Venture Capital Partners")
  slug:
  String,           // unique slug for portal URL (lowercase, URL-safe, unique index)
  created_at: int,   // UNIX timestamp
  created_by: ObjectId, // user who created the tenant (admin or
  self-registration)
  stats: {           // optional aggregated statistics for dashboards
    job_count: int,
    company_count: int,
    location_count: int
  }
}
```

Create a unique index on `slug` to guarantee uniqueness.

1. **users collection changes** – add role `vc_admin` and field `tenant_id` that references the tenant. Users with role `vc_admin` can upload jobs for their tenant. Additional roles (`recruiter`, `analyst`, `candidate`) remain unchanged. The `admin` role remains global.

2. **jobs collection changes** - add `company_name`, `application_url` and `company_website` fields. Remove `city_canonical` (the canonical value can be generated dynamically). Ensure jobs have `tenant_id` and index `(tenant_id, external_job_id)` remains as before.

3. **CSV template** - the job import will expect a CSV with the following headers:

```
external_job_id,title,description,city,must_have,nice_to_have,profession,occupation_field,remote
1234,Senior Full-Stack Engineer,Build next-generation products,Tel Aviv,"React|Node.js|TypeScript","GraphQL|AWS",Engineering,Software,true,TechFlow AI,,5678,Product Manager,Lead product development,New York,,,Product,Management,false,DataVision,,
```

must_have and *nice_to_have* are pipe-delimited lists. *remote* is `true` or `false`.

3. Backend API implementation

3.1. Models and utilities

Create a new file `backend/app/api/routers/tenants.py` and define Pydantic models for tenant and CSV upload:

```
# backend/app/api/routers/tenants.py
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File
from pydantic import BaseModel, EmailStr
from typing import List
from bson import ObjectId
import csv, io, time, secrets, re
from ..core.database import db
from ..core import security
from ..dependencies import get_current_user, require_admin, require_role

router = APIRouter(prefix="/tenants", tags=["tenants"])

class TenantCreate(BaseModel):
    name: str
    admin_name: str
    admin_email: EmailStr
    admin_password: str

class UserOut(BaseModel):
    user_id: str
    email: EmailStr
    role: str
    tenant_id: str | None = None

class TenantOut(BaseModel):
    tenant_id: str
```

```

slug: str
name: str

class CSVImportResponse(BaseModel):
    inserted_count: int
    replaced_count: int

# Utility to generate a URL-safe slug
_slug_re = re.compile(r'^[a-z0-9]+')
def generate_slug(name: str) -> str:
    base = _slug_re.sub('-', name.lower()).strip('-')
    # ensure uniqueness
    slug = base
    counter = 1
    while db.tenants.find_one({"slug": slug}):
        slug = f"{base}-{counter}"; counter += 1
    return slug

```

3.2. Registration and tenant creation

Expose a public endpoint to create a tenant and the first user. It returns the portal URL.

```

@router.post("", response_model=TenantOut)
def create_tenant(payload: TenantCreate):
    # Check if user already exists
    if db.users.find_one({"email": payload.admin_email}):
        raise HTTPException(status_code=409, detail="email_exists")
    # Create tenant
    slug = generate_slug(payload.name)
    tenant_doc = {
        "name": payload.name,
        "slug": slug,
        "created_at": int(time.time()),
    }
    tenant_id = db.tenants.insert_one(tenant_doc).inserted_id
    # Create admin user within the tenant
    user_doc = {
        "name": payload.admin_name,
        "email": payload.admin_email,
        "password_hash": security.hash_password(payload.admin_password),
        "role": "vc_admin",
        "tenant_id": tenant_id,
        "created_at": int(time.time()),
    }
    db.users.insert_one(user_doc)
    return TenantOut(tenant_id=str(tenant_id), slug=slug, name=payload.name)

```

3.3. Admin APIs

Add endpoints to list, create and delete users. Only global admins (role `admin`) may call these endpoints.

```
@router.get("/users", dependencies=[Depends(require_admin)])
def list_users():
    items = []
    for u in db.users.find({}).sort("created_at", -1):
        items.append(UserOut(user_id=str(u["_id"]), email=u["email"],
                             role=u["role"], tenant_id=str(u.get("tenant_id")) if u.get("tenant_id") else
                             None))
    return {"items": items}

class UserCreateAdmin(BaseModel):
    name: str
    email: EmailStr
    password: str
    role: str
    tenant_id: str | None = None

@router.post("/users", dependencies=[Depends(require_admin)])
def create_user_admin(payload: UserCreateAdmin):
    if db.users.find_one({"email": payload.email}):
        raise HTTPException(status_code=409, detail="email_exists")
    if payload.role not in ("admin", "vc_admin", "recruiter", "analyst",
                           "candidate"):
        raise HTTPException(status_code=400, detail="invalid_role")
    tenant_obj = ObjectId(payload.tenant_id) if payload.tenant_id else None
    doc = {
        "name": payload.name,
        "email": payload.email,
        "password_hash": security.hash_password(payload.password),
        "role": payload.role,
        "tenant_id": tenant_obj,
        "created_at": int(time.time()),
    }
    uid = db.users.insert_one(doc).inserted_id
    return {"user_id": str(uid)}

@router.delete("/users/{user_id}", dependencies=[Depends(require_admin)])
def delete_user(user_id: str):
    obj_id = ObjectId(user_id)
    result = db.users.delete_one({"_id": obj_id})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail="user_not_found")
    return {"success": True}
```

3.4. CSV upload for portfolio jobs

VC admins and global admins can upload a CSV to replace all jobs of a tenant. The endpoint accepts a file and parses it.

```
from fastapi import status

@router.post("/{tenant_id}/jobs/upload", response_model=CSVImportResponse)
def upload_jobs_csv(tenant_id: str, file: UploadFile = File(...),
current_user=Depends(get_current_user)):
    # Only allow if current user is admin or vc_admin of this tenant
    if current_user["role"] not in ("admin", "vc_admin"):
        raise HTTPException(status_code=403, detail="forbidden")
    if current_user["role"] == "vc_admin" and str(current_user.get("tenant_id")) !=
tenant_id:
        raise HTTPException(status_code=403, detail="wrong_tenant")
    # Read CSV
    content = await file.read()
    reader = csv.DictReader(io.StringIO(content.decode()))
    jobs = []
    required = ["external_job_id", "title", "description"]
    for row in reader:
        if not all(row.get(k) for k in required):
            continue # skip incomplete rows
        must = [s.strip().lower() for s in row.get("must_have", "").split("|") if
s.strip()]
        nice = [s.strip().lower() for s in row.get("nice_to_have", "").split("|") if
s.strip()]
        merged = list(dict.fromkeys(must + nice)) # unique order preserving
        jobs.append({
            "tenant_id": ObjectId(tenant_id),
            "external_job_id": row["external_job_id"],
            "title": row["title"],
            "job_description": row["description"],
            "city": row.get("city"),
            "requirements": {
                "must_have_skills": [{"name": s} for s in must],
                "nice_to_have_skills": [{"name": s} for s in nice],
            },
            "job_requirements": must + nice,
            "skill_set": merged,
            "profession": row.get("profession"),
            "occupation_field": row.get("occupation_field"),
            "remote": row.get("remote", "false").lower() == "true",
            "company_name": row.get("company_name"),
            "application_url": row.get("application_url"),
            "company_website": row.get("company_website"),
            "created_at": int(time.time()),
            "updated_at": int(time.time()),
        })
```

```

    })
    # Replace existing jobs for tenant
    replaced = db.jobs.delete_many({"tenant_id":
ObjectId(tenant_id)}).deleted_count
    if jobs:
        db.jobs.insert_many(jobs)
    return CSVImportResponse(inserted_count=len(jobs), replaced_count=replaced)

```

3.5. Public portal endpoint

A public, unauthenticated endpoint returns the tenant's metadata and jobs for use in the portal UI. It searches by slug.

```

@router.get("/public/portal/{slug}")
def get_public_portal(slug: str):
    tenant = db.tenants.find_one({"slug": slug})
    if not tenant:
        raise HTTPException(status_code=404, detail="portal_not_found")
    t_id = tenant["_id"]
    jobs = []
    for j in db.jobs.find({"tenant_id": t_id}).sort("created_at", -1):
        jobs.append({
            "job_id": str(j["_id"]),
            "title": j.get("title"),
            "company_name": j.get("company_name", ""),
            "description": j.get("job_description", ""),
            "requirements": j.get("skill_set", []),
            "location": j.get("city", "Remote"),
            "remote": j.get("remote", False),
            "application_url": j.get("application_url"),
            "company_website": j.get("company_website"),
        })
    return {
        "name": tenant["name"],
        "slug": tenant["slug"],
        "stats": {
            "job_count": len(jobs),
            "company_count": len({j["company_name"] for j in jobs if
j["company_name"]}),
            "location_count": len({j["location"] for j in jobs}),
        },
        "jobs": jobs,
    }

```

3.6. Integrating routers

In `backend/app/main.py`, import and include the new router:

```
from .api.routers import tenants as tenants_router
# ... existing code ...
app.include_router(tenants_router.router)
```

Also update `backend/app/core/database.py` to create indexes for the new collection:

```
# after existing index creation
try:
    db.tenants.create_index("slug", unique=True)
except Exception as e:
    print(f"Tenant index creation error: {e}")
```

4. Frontend implementation

The existing `frontend` folder uses React with React Router. We extend it as follows.

4.1. API helper updates

Update `frontend/src/api.ts` to include new endpoints:

```
// existing imports ...
export async function apiPost(path: string, body: any) {
    const token = localStorage.getItem('token')
    const res = await fetch(`${API_BASE}${path}`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            ...(token ? { Authorization: `Bearer ${token}` } : {}),
        },
        body: JSON.stringify(body),
    })
    if (!res.ok) throw new Error(await res.text())
    return res.json()
}

export async function apiUpload(path: string, file: File) {
    const token = localStorage.getItem('token')
    const formData = new FormData()
    formData.append('file', file)
    const res = await fetch(`${API_BASE}${path}`, {
        method: 'POST',
        headers: { ...(token ? { Authorization: `Bearer ${token}` } : {}), },
        body: formData,
    })
    if (!res.ok) throw new Error(await res.text())
}
```



```

    return res.json()
  }

```

4.2. Registration Wizard (/registration)

Create `frontend/src/pages/RegistrationPage.tsx`. The wizard collects the fund name, admin name, email, password and optional details (portfolio size, ATS mix). After submitting, it calls `POST /tenants` and displays the generated slug and portal link.

```

import React, { useState } from 'react'
import { apiPost } from '../api'
import { useNavigate } from 'react-router-dom'

const RegistrationPage: React.FC = () => {
  const [step, setStep] = useState(1)
  const [form, setForm] = useState({
    name: '',
    admin_name: '',
    admin_email: '',
    admin_password: '',
    portcos: '',
    ats: '',
    focus: '',
  })
  const [result, setResult] = useState<any>(null)
  const handleChange = (e: React.ChangeEvent<HTMLInputElement |
HTMLTextAreaElement>) => {
    setForm({ ...form, [e.target.name]: e.target.value })
  }
  const handleNext = () => setStep(step + 1)
  const handlePrev = () => setStep(step - 1)
  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    const tenant = await apiPost('/tenants', {
      name: form.name,
      admin_name: form.admin_name,
      admin_email: form.admin_email,
      admin_password: form.admin_password,
    })
    setResult(tenant)
  }
  if (result) {
    return (
      <div style={{ padding: 24 }}>
        <h2>Registration complete</h2>
        <p>Your portal is ready at:</p>
        <a href={` /portal/${result.slug}`}
target="_blank">{window.location.origin}/portal/{result.slug}</a>
      </div>
    )
  }

```

```

    )
  }
  return (
    <div style={{ maxWidth: 600, margin: '0 auto', padding: 24 }}>
      <h2>Create Your Account</h2>
      <form onSubmit={handleSubmit}>
        {step === 1 && (
          <div>
            <label>Fund/Firm Name</label>
            <input name="name" value={form.name} onChange={handleChange} required />
            <label>Your Name</label>
            <input name="admin_name" value={form.admin_name} onChange={handleChange}
required />
            <label>Email</label>
            <input name="admin_email" type="email" value={form.admin_email}
onChange={handleChange} required />
            <label>Password</label>
            <input name="admin_password" type="password" value={form.admin_password}
onChange={handleChange} required />
          </div>
        )}
        {step === 2 && (
          <div>
            <label>Portfolio Companies (optional)</label>
            <input name="portcos" value={form.portcos} onChange={handleChange} />
            <label>ATS Mix (optional)</label>
            <input name="ats" value={form.ats} onChange={handleChange} />
            <label>Key Focus Areas (optional)</label>
            <textarea name="focus" value={form.focus} onChange={handleChange}></
textarea>
          </div>
        )}
        <div style={{ marginTop: 16 }}>
          {step > 1 && <button type="button" onClick={handlePrev}>Back</button>}
          {step < 2 && <button type="button" onClick={handleNext}>Next</button>}
          {step === 2 && <button type="submit">Register</button>}
        </div>
      </form>
    </div>
  )
}
export default RegistrationPage

```

4.3. Admin pages

Add a route `/admin/users` that renders a table of all users with their role, email, tenant and actions to delete. Create `frontend/src/pages/AdminUsersPage.tsx`:

```

import React, { useEffect, useState } from 'react'
import { apiGet, apiPost } from '../api'

const AdminUsersPage: React.FC = () => {
  const [users, setUsers] = useState<any[]>([])
  const [error, setError] = useState<string | null>(null)
  const [creating, setCreating] = useState(false)
  const [form, setForm] = useState({ name: '', email: '', password: '', role:
'vc_admin', tenant_id: '' })
  useEffect(() => {
    apiGet('/tenants/users').then((data) => setUsers(data.items)).catch((err) =>
setError(err.message))
  }, [])
  const handleChange = (e: any) => setForm({ ...form, [e.target.name]:
e.target.value })
  const createUser = async () => {
    await apiPost('/tenants/users', form)
    setCreating(false)
    const data = await apiGet('/tenants/users')
    setUsers(data.items)
  }
  return (
    <div style={{ padding: 24 }}>
      <h2>User Management</h2>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <button onClick={() => setCreating(!creating)}>+ New User</button>
      {creating && (
        <div style={{ marginTop: 12 }}>
          <input placeholder="Name" name="name" value={form.name}
onChange={handleChange} />
          <input placeholder="Email" name="email" type="email" value={form.email}
onChange={handleChange} />
          <input placeholder="Password" name="password" type="password"
value={form.password} onChange={handleChange} />
          <select name="role" value={form.role} onChange={handleChange}>
            <option value="vc_admin">VC Admin</option>
            <option value="recruiter">Recruiter</option>
            <option value="analyst">Analyst</option>
            <option value="candidate">Candidate</option>
            <option value="admin">Admin</option>
          </select>
          <input placeholder="Tenant ID (for VC roles)" name="tenant_id"
value={form.tenant_id} onChange={handleChange} />
          <button onClick={createUser}>Create</button>
        </div>
      )}
    <table style={{ width: '100%', marginTop: 24 }}>
      <thead><tr><th>Email</th><th>Role</th><th>Tenant</th></tr></thead>
      <tbody>

```

```

    {users.map((u) => (
      <tr key={u.user_id}>
        <td>{u.email}</td>
        <td>{u.role}</td>
        <td>{u.tenant_id || '-'}</td>
      </tr>
    ))}
  </tbody>
</table>
</div>
)
}
export default AdminUsersPage

```

You will also need an `AdminTenantPage.tsx` that allows uploading a CSV for a specific tenant. It should read a file input and call `apiUpload`:

```

import React, { useState } from 'react'
import { apiUpload } from '../api'
import { useParams } from 'react-router-dom'

const AdminTenantPage: React.FC = () => {
  const { tenantId } = useParams<{ tenantId: string }>()
  const [file, setFile] = useState<File | null>(null)
  const [status, setStatus] = useState('')
  const upload = async () => {
    if (file && tenantId) {
      setStatus('Uploading...')
      try {
        const res = await apiUpload(`/tenants/${tenantId}/jobs/upload`, file)
        setStatus(`Uploaded ${res.inserted_count} jobs (replaced $
{res.replaced_count})`)
      } catch (err: any) {
        setStatus('Error: ' + err.message)
      }
    }
  }
  return (
    <div style={{ padding: 24 }}>
      <h2>Manage Tenant {tenantId}</h2>
      <p>Upload a CSV to replace existing jobs.</p>
      <input type="file" accept=".csv" onChange={(e) => setFile(e.target.files?.[0]
|| null)} />
      <button disabled={!file} onClick={upload}>Upload</button>
      <div>{status}</div>
    </div>
  )
}

```

```

}
export default AdminTenantPage

```

4.4. Portal page

Create a dedicated page that displays jobs for a tenant using the design from `demo-portal.html`. Place it at `frontend/src/pages/PortalPage.tsx` and mount it via route `/portal/:slug`.

```

import React, { useEffect, useState } from 'react'
import { useParams } from 'react-router-dom'

const PortalPage: React.FC = () => {
  const { slug } = useParams<{ slug: string }>()
  const [data, setData] = useState<any>(null)
  const [query, setQuery] = useState('')
  const [location, setLocation] = useState('')
  const [company, setCompany] = useState('')
  const [type, setType] = useState('')
  useEffect(() => {
    fetch(`/api/tenants/public/portal/${slug}`).then((res) =>
      res.json()).then(setData)
  }, [slug])
  if (!data) return <p>Loading...</p>
  let jobs = data.jobs
  // filter
  jobs = jobs.filter((j: any) => {
    const q = query.toLowerCase()
    return (!query || j.title.toLowerCase().includes(q) ||
      j.company_name.toLowerCase().includes(q) ||
      j.description.toLowerCase().includes(q) || j.requirements.join('
').toLowerCase().includes(q)) &&
      (!location || j.location === location) &&
      (!company || j.company_name === company) &&
      (!type || (type === 'remote' && j.remote) || (type === 'onsite' && !j.remote))
  })
  const locations = Array.from(new Set(data.jobs.map((j: any) =>
    j.location).filter(Boolean))).sort()
  const companies = Array.from(new Set(data.jobs.map((j: any) =>
    j.company_name).filter(Boolean))).sort()
  return (
    <div style={{ background: '#0f172a', color: '#e5e7eb', minHeight: '100vh',
paddingBottom: 40 }}>
      <header style={{ background: 'rgba(17,24,39,.90)', borderBottom: '1px solid
#1f2937', padding: '16px 0' }}>
        <div style={{ maxWidth: 1200, margin: '0 auto', display: 'flex',
justifyContent: 'space-between', alignItems: 'center', padding: '0 20px' }}>
          <div style={{ display: 'flex', gap: 12, alignItems: 'center' }}>
            <div style={{ width: 36, height: 36, borderRadius: 8, background: 'linear-
gradient(135deg,#10b981,#06b6d4)', display: 'flex', alignItems: 'center',

```

```

justifyContent: 'center', color: '#fff', fontWeight: 700 }}>{data.name.slice(0,
2).toUpperCase()}</div>
    <div>
      <div>{data.name}</div>
      <div style={{ fontSize: 12, fontWeight: 400, color: '#94a3b8' }}>Portfolio
Jobs</div>
    </div>
  </div>
  <span style={{ color: '#94a3b8', fontSize: 14 }}>Apply once. Be seen by all
portfolio companies.</span>
</div>
</header>
<main style={{ maxWidth: 1200, margin: '0 auto', padding: '40px 20px' }}>
  <section style={{ textAlign: 'center', marginBottom: 32 }}>
    <h1 style={{ fontSize: 42, fontWeight: 700, background: 'linear-
gradient(135deg,#10b981,#06b6d4)', WebkitBackgroundClip: 'text', color:
'transparent' }}>Join Our Portfolio Ecosystem</h1>
    <p style={{ fontSize: 18, color: '#94a3b8', maxWidth: 600, margin: '0 auto' }}>
>Discover exceptional opportunities across our high-growth startups and
scale-ups.</p>
    <div style={{ display: 'flex', justifyContent: 'center', gap: 40, marginTop:
24 }}>
      <div style={{ textAlign: 'center' }}>
        <span style={{ fontSize: 28, fontWeight: 700, color: '#10b981', display:
'block' }}>{data.stats.job_count}</span>
        <span style={{ fontSize: 14, color: '#94a3b8' }}>Open Positions</span>
      </div>
      <div style={{ textAlign: 'center' }}>
        <span style={{ fontSize: 28, fontWeight: 700, color: '#10b981', display:
'block' }}>{data.stats.company_count}</span>
        <span style={{ fontSize: 14, color: '#94a3b8' }}>Companies</span>
      </div>
      <div style={{ textAlign: 'center' }}>
        <span style={{ fontSize: 28, fontWeight: 700, color: '#10b981', display:
'block' }}>{data.stats.location_count}</span>
        <span style={{ fontSize: 14, color: '#94a3b8' }}>Cities</span>
      </div>
    </div>
  </section>
  <section>
    <div style={{ display: 'flex', gap: 16, flexWrap: 'wrap', marginBottom: 24 }}>
      <div style={{ display: 'flex', flexDirection: 'column' }}>
        <label style={{ fontSize: 13, fontWeight: 600 }}>Search</label>
        <input style={{ background: '#0b1220', border: '1px solid #334155',
borderRadius: 8, color: '#e5e7eb', padding: '12px 16px', minWidth: 280 }}
value={query} onChange={(e) => setQuery(e.target.value)}
placeholder="Job title, skills, company..." />
      </div>
      <div style={{ display: 'flex', flexDirection: 'column' }}>
        <label style={{ fontSize: 13, fontWeight: 600 }}>Location</label>

```

```

    <select style={{ background: '#0b1220', border: '1px solid #334155',
borderRadius: 8, color: '#e5e7eb', padding: '12px 16px' }}
    value={location} onChange={(e) => setLocation(e.target.value)}>
    <option value="">All locations</option>
    {locations.map((loc) => <option key={loc} value={loc}>{loc}</option>)}
    </select>
  </div>
  <div style={{ display: 'flex', flexDirection: 'column' }}>
    <label style={{ fontSize: 13, fontWeight: 600 }}>Company</label>
    <select style={{ background: '#0b1220', border: '1px solid #334155',
borderRadius: 8, color: '#e5e7eb', padding: '12px 16px' }}
    value={company} onChange={(e) => setCompany(e.target.value)}>
    <option value="">All companies</option>
    {companies.map((c) => <option key={c} value={c}>{c}</option>)}
    </select>
  </div>
  <div style={{ display: 'flex', flexDirection: 'column' }}>
    <label style={{ fontSize: 13, fontWeight: 600 }}>Type</label>
    <select style={{ background: '#0b1220', border: '1px solid #334155',
borderRadius: 8, color: '#e5e7eb', padding: '12px 16px' }}
    value={type} onChange={(e) => setType(e.target.value)}>
    <option value="">All types</option>
    <option value="remote">Remote</option>
    <option value="onsite">On-site</option>
    </select>
  </div>
</div>
<div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', marginBottom: 16 }}>
  <h2 style={{ fontSize: 24, fontWeight: 700 }}>Open Positions</h2>
  <div style={{ fontSize: 14, color: '#94a3b8', padding: '8px 16px',
background: 'rgba(16,185,129,0.1)', borderRadius: 20, border: '1px solid
rgba(16,185,129,0.2)' }}>
    {jobs.length} {jobs.length === 1 ? 'position' : 'positions'}
  </div>
</div>
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-
fill,minmax(340px,1fr))', gap: 20 }}>
  {jobs.map((job: any) => (
    <div key={job.job_id} style={{ background: '#111827', border: '1px solid
#1f2937', borderRadius: 16, padding: 20, display: 'flex', flexDirection:
'column', gap: 16 }}>
      <div style={{ display: 'flex', alignItems: 'center', gap: 12 }}>
        <div style={{ width: 44, height: 44, borderRadius: 10, background:
'linear-gradient(135deg,#10b981,#06b6d4)', display: 'flex', alignItems:
'center', justifyContent: 'center', color: '#fff', fontWeight: 700 }}
        >{(job.company_name || '?').slice(0,2).toUpperCase()}</div>
        <div>
          <h3 style={{ margin: 0, fontSize: 18, color: '#e5e7eb' }}>{job.title}</
h3>

```

```

        <p style={{ margin: 0, fontSize: 14, color: '#94a3b8' }}
>{job.company_name}</p>
      </div>
    </div>
    <div style={{ fontSize: 13, color: '#94a3b8', display: 'flex', gap: 12,
flexWrap: 'wrap' }}>
      <span>📍 {job.location}</span>
      {job.remote && <span style={{ background: 'rgba(16,185,129,0.15)',
color: '#10b981', padding: '4px 8px', borderRadius: 6, fontSize: 11, fontWeight:
600 }}>Remote</span>}
    </div>
    <p style={{ color: '#94a3b8', fontSize: 14, margin: 0 }}>{job.description}
</p>
    <div style={{ display: 'flex', flexWrap: 'wrap', gap: 6 }}>
      {job.requirements.map((s: string) => <span key={s} style={{ background:
'rgba(255,255,255,0.06)', border: '1px solid rgba(255,255,255,0.1)', color:
'#e5e7eb', padding: '4px 8px', borderRadius: 6, fontSize: 12, fontWeight: 500 }}
>{s}</span>)}
    </div>
    <button style={{ marginTop: 'auto', background: 'linear-
gradient(135deg,#10b981,#06b6d4)', color: '#fff', border: 'none', borderRadius:
10, padding: '12px 20px', fontSize: 14, fontWeight: 600, cursor: 'pointer' }}
onClick={() => {
      const url = job.application_url || job.company_website || ''
      if (url) window.open(url, '_blank')
    }}>Apply Now 📄</button>
  </div>
))}
{jobs.length === 0 && <div style={{ gridColumn: '1 / -1', textAlign:
'center', padding: 40, color: '#94a3b8' }}>No positions found</div>}
</div>
</section>
</main>
<footer style={{ borderTop: '1px solid #1f2937', padding: '40px 0',
textAlign: 'center', color: '#94a3b8', fontSize: 14 }}>
  Powered by PTX 📄 Portfolio Talent Exchange
</footer>
</div>
)
}
export default PortalPage

```

4.5. Routing

Update your router in `App.tsx` to include new pages:

```

// import new pages ...
import RegistrationPage from './pages/RegistrationPage'
import PortalPage from './pages/PortalPage'

```



```
import AdminUsersPage from './pages/AdminUsersPage'
import AdminTenantPage from './pages/AdminTenantPage'

<Routes>
  {/* existing routes */}
  <Route path="/registration" element={<RegistrationPage />} />
  <Route path="/portal/:slug" element={<PortalPage />} />
  <Route path="/admin/users" element={<AdminUsersPage />} />
  <Route path="/admin/tenants/:tenantId" element={<AdminTenantPage />} />
</Routes>
```

5. CSV template file

Provide a file `job_upload_template.csv` in the root of the repository for admins and VC users to download and fill. The first line contains headers and subsequent lines demonstrate valid values:

```
external_job_id,title,description,city,must_have,nice_to_have,profession,occupation_field,remo
1234,Senior Full-Stack Engineer,Build next-generation products,Tel
Aviv,"React|Node.js|TypeScript","GraphQL|
AWS",Engineering,Software,true,TechFlow AI,,
5678,Product Manager,Lead product development,New
York,,,Product,Management,false,DataVision,,
```

Place this file in the `public` or `static` directory of the frontend so it can be downloaded.

6. Step-by-step instructions for VS Code Copilot

1. **Pull latest changes** of the PTX repository and create a new feature branch, e.g. `feature/vc-portal`.
2. **Backend changes:**
3. Create a new file `backend/app/api/routers/tenants.py` and paste the code from sections 3.1–3.5 above.
4. Update `backend/app/main.py` to import and include the new router.
5. Modify `backend/app/core/database.py` to create the `slug` index on the new `tenants` collection.
6. **Add job fields:** update the `jobs` router (if necessary) and any Pydantic models to include `company_name`, `application_url` and `company_website` so they can be stored and returned.
7. **Run backend tests** and start the FastAPI development server to verify that `/tenants` endpoints work and that JWT roles are enforced correctly.
8. **Frontend changes:**
9. Update `frontend/src/api.ts` with the `apiPost` and `apiUpload` helpers shown above.
10. Create new pages `RegistrationPage.tsx`, `PortalPage.tsx`, `AdminUsersPage.tsx` and `AdminTenantPage.tsx` in `frontend/src/pages/` and paste the corresponding code.
11. Update the router in `frontend/src/App.tsx` to add routes for `/registration`, `/portal/:slug`, `/admin/users` and `/admin/tenants/:tenantId`.
12. Place the provided `job_upload_template.csv` in the frontend's `public/static` folder.

13. **Run the React development server** (`npm start` or `pnpm dev`) and navigate to `/registration` to test the wizard. Register a new tenant and verify that a `slug` is returned and that the portal is accessible.
14. **As an admin**, visit `/admin/users` to list users and create new VC admins or recruiters. Click a tenant ID to navigate to `/admin/tenants/{tenantId}` and upload a CSV using the template.
15. **Visit the portal** using the generated slug (e.g., `/portal/venture-capital-partners`) to ensure jobs display correctly and filters work. Compare the UI with the demo template and adjust styles as needed.
16. **Commit changes** and open a pull request for code review.

Conclusion

By extending the existing PTX backend with a `tenants` collection, additional user roles and CSV import functionality, and by building new React pages that reuse the dark-themed styling from the demo templates ⁸, we deliver a production-ready solution for venture capital partners. Administrators gain control over user management and data imports, while each VC fund receives a unique job portal accessible via a slug. The provided CSV template and step-by-step instructions ensure a smooth implementation process and align the real product with the polished design of the demo.

¹ ² `auth.py`

<https://github.com/letsgetstartup/PTX/blob/main/backend/app/api/routers/auth.py>

³ ⁴ `jobs.py`

<https://github.com/letsgetstartup/PTX/blob/main/backend/app/api/routers/jobs.py>

⁵ `database.py`

<https://github.com/letsgetstartup/PTX/blob/main/backend/app/core/database.py>

⁶ `AdminDashboard.tsx`

<https://github.com/letsgetstartup/PTX/blob/main/frontend/src/pages/AdminDashboard.tsx>

⁷ `RecruiterJobsPage.tsx`

<https://github.com/letsgetstartup/PTX/blob/main/frontend/src/pages/RecruiterJobsPage.tsx>

⁸ ⁹ ¹⁰ `demo-portal.html`

<https://github.com/letsgetstartup/amis-talent-matching/blob/main/frontend/public/demo-portal.html>